

Furiosa RNGD

코드 생성 모델 벤치마크

측정 기준 · 5 개 테스트 방법 · 과정 · 결과 — RNGD 2- 카드 환경에서 서빙 가능한 4 개 모델 검증

핵심 결론

코드 생성 후보 = Llama-3.1-8B-Instruct

단일 55 tok/s · 동시성 c128 까지 요청 실패 0 · RNGD 1 카드 — 단, 강력 후보 32B·70B 는 4 카드 필요로 제외돼 모델 간 비교는 불성립

측정 테스트

5 종

속도 · 동시성 · serve 옵션 · SWE-bench · 임베딩

측정 모델

4/7 종

2 카드 서빙 가능 모델만

자동화

1 YAML

모델 추가 시 전 과정 자동

무엇을 , 어떤 기준으로 검증했나

RNGD prebuilt 모델 중 코드 생성에 가장 적합한 모델을 4 개 축으로 평가

검증 목표

Furiosa RNGD 에서 제공되는 prebuilt 모델을 대상으로 코드 생성 작업에 가장 적합한 모델을 가려내고 , 그 판단 근거를 정량 측정으로 확보한다 .

01

토큰 생성 속도

속도

단일 요청에서 첫 토큰 지연 (TTFT) 과
초당 생성 토큰 수 (tok/s) 를 측정

해당 테스트 · 테스트 1

02

동시 접속 확장성

동시성 · 배치

동시 요청을 늘리며 합산 처리량 · 요청당
지연 · 실패율의 변화를 측정

해당 테스트 · 테스트 2 · 3

03

SWE-bench

정확도

실제 GitHub 이슈를 코드 패치로
해결하는 정확도를 테스트 통과로 채점

해당 테스트 · 테스트 4

04

서빙 · 자동화

환경

furiosa-llm 서빙 구성과 신규 모델
자동 분석 파이프라인 구축

해당 테스트 · 테스트 5 · 전반

측정 환경 — furiosa-llm OpenAI 호환 서버

vLLM 은 RNGD 에 직접 붙지 않음 → furiosa-llm 이 vLLM 호환 OpenAI API 를 제공

측정 클라이언트

httpx 비동기
/v1/chat/completions 호출



furiosa-llm serve

OpenAI 호환 API 서버
(vLLM 과 동일 스펙)



RNGD NPU

prebuilt 아티팩트 실행
npu0 / npu1

서빙

furiosa-llm serve

각 모델을 OpenAI 호환 서버로 기동 . 측정 도구는 vLLM
과 동일한 /v1 endpoint 를 그대로 사용 .

측정

스트리밍 클라이언트

응답을 토큰 단위 스트림으로 받아 TTFT · 토큰 간 지연을
실측 . 동시 요청은 비동기로 발생 .

채점

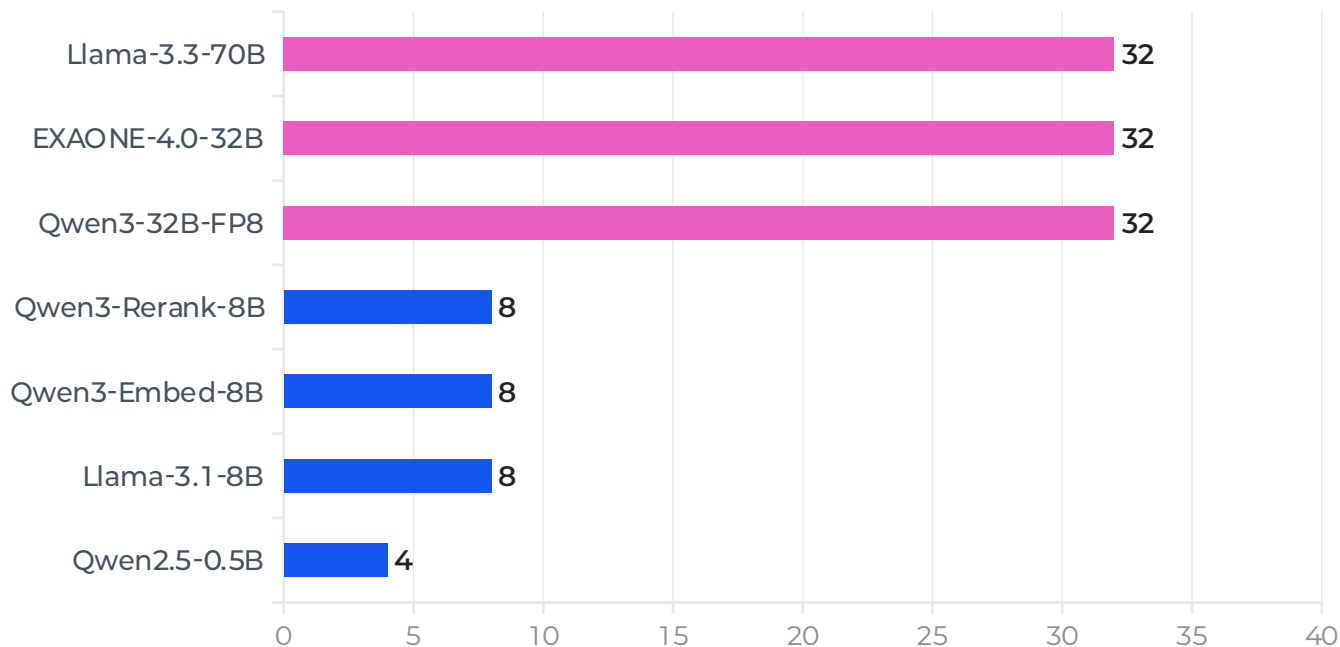
Docker harness

SWE-bench 채점에만 Docker 사용 — 인스턴스별 격리
컨테이너에서 테스트 실행 .

7종 중 4종만 2-카드에서 측정 가능

prebuilt 아티팩트의 tensor-parallel 크기가 필요 NPU 수를 고정한다

모델별 필요 PE (RNGD 1 카드 = 8 PE)



파랑 = 2 카드 (16 PE) 로 측정 가능 · 분홍 = 4 카드 필요 → 측정 불가

측정 대상 4 종

Qwen2.5-0.5B · Llama-3.1-8B · Qwen3-Embedding-8B
· Qwen3-Reranker-8B — 모두 1 카드 (≤8 PE)

측정 제외 3 종

Qwen3-32B-FP8 · EXAONE-4.0-32B-FP8 · Llama-3.3-70B — 아티팩트가 tp=32(4 카드) 로 컴파일

코드 생성 후보

측정 가능 생성 모델은 Llama-3.1-8B 단독 (0.5B 는 검증용) →
모델 간 비교 불성립

측정 파이프라인 — 모델 추가만으로 자동화

configs/models.yaml 에 모델을 등록하면 서버 기동부터 리포트까지 자동 실행



02-04 단계는 모델마다 반복

한 모델의 서버를 띄워 테스트 1·2·4 를 측정하고, 테스트 3(memswep)은 serve 옵션을 바꿔 서버를 재기동하며 측정한다. 측정이 끝나면 서버를 내리고 다음 모델로 — 7 종이든 새 모델이든 동일 흐름. 측정 도구 코드는 수정 불필요, models.yaml 만 바꾸면 된다.

측정 테스트 5 종 한눈에

각 모델에 대해 아래 5 개 테스트를 순차 수행 — 이후 슬라이드는 테스트 번호 순

테스트 1

토큰 생성 속도

단일 요청 스트리밍으로 첫 토큰
지연 · 생성 속도 측정

지표 TTFT · tok/s

테스트 2

동시성 스케일링

동시 요청을 1→128 로 늘리며
처리량 · 지연 변화 측정

지표 합산 TPS · 실패율

테스트 3

serve 옵션

serve 인자를 바꿔가며 처리량
영향 측정 (memsweep)

지표 조합별 합산 TPS

테스트 4

SWE-bench

실제 GitHub 이슈를 코드 패치로
해결, 테스트 통과 채점

지표 resolved %

테스트 5

임베딩 · 리랭커

검색 보조 모델의 배치별 처리량
측정

지표 inputs/s

테스트 1 — 토큰 생성 속도

단일 요청 스트리밍으로 첫 토큰 지연 (TTFT) · 생성 속도 (tok/s) 측정

무엇을

동시 사용자 1 명일 때 체감 속도 — 코드 자동완성 · 대화형 보조의 핵심 지표 .

입력

고정 프롬프트를 /v1/chat/completions 에 스트리밍 요청 .
max_tokens 256, 워밍업 5 + 측정 50 회 .

출력 · 분석

요청별 TTFT · ITL (토큰 간 간격) · 출력 tok/s 를 p50/p95 로 집계 .

결과

Llama-3.1-8B-Instruct

55.2 tok/s

출력 토큰 처리량 (단일 요청)

TTFT 32 ms ITL 18.2 ms

코드 생성 후보 · context 128K

Qwen2.5-0.5B-Instruct

85.3 tok/s

출력 토큰 처리량 (단일 요청)

TTFT 31 ms ITL 11.8 ms

파이프라인 검증용 (smoke)

테스트 2 — 동시성 스케일링

동시 요청을 1→128 로 늘리며 합산 처리량 · 지연 변화 측정

무엇을

동시 접속자 수에 따른 처리량 · 지연 — 서버 운영 조건 산정의 근거.

입력

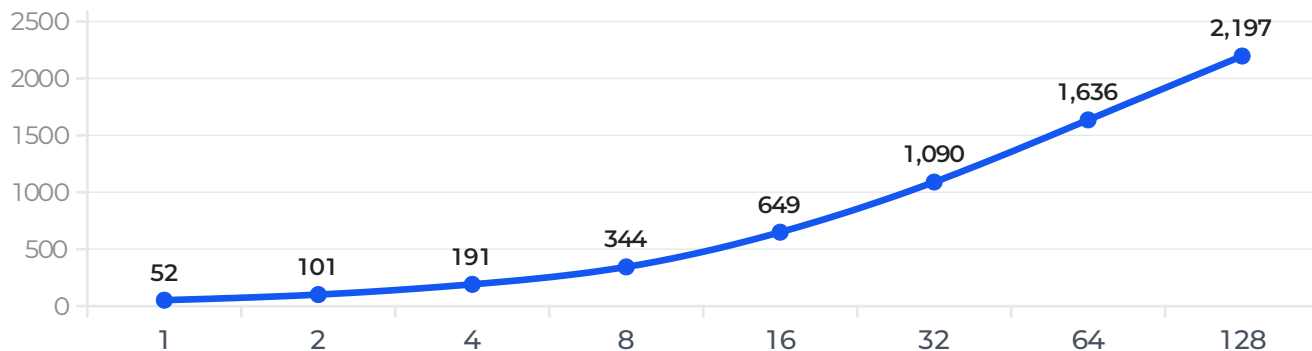
동시성 {1·2·4·8·16·32·64·128} × 프롬프트 길이 {256·1024·4096} 매트릭스.

출력 · 분석

셀별 합산 처리량 · 요청당 속도 · TTFT · 실패 수 → 효율 배치점 도출.

결과

동시성별 합산 처리량 (tok/s) — Llama-3.1-8B



스케일링

처리량 약 20 배

c1 52 → c128 2,197 tok/s. 동시성을 올릴수록 합산 처리량 비례 이상 증가.

지연

TTFT 1.8s @c128

첫 토큰 지연 0.04→1.85s, 요청당 속도 54→18 tok/s. 전 구간 실패 0 건.

판단

효율 배치점 c32

제곱 속도 (35 tok/s+) 유지하는 1 카드 권장 상한은 동시 32 명.

테스트 3 — serve 옵션 튜닝 (memsweep)

서버 기동 옵션을 한 번에 하나씩 바꿔 (OFAT) 처리량 영향 측정

무엇을

KV cache·배치 관련 serve 옵션이 처리량에 주는 영향.

입력

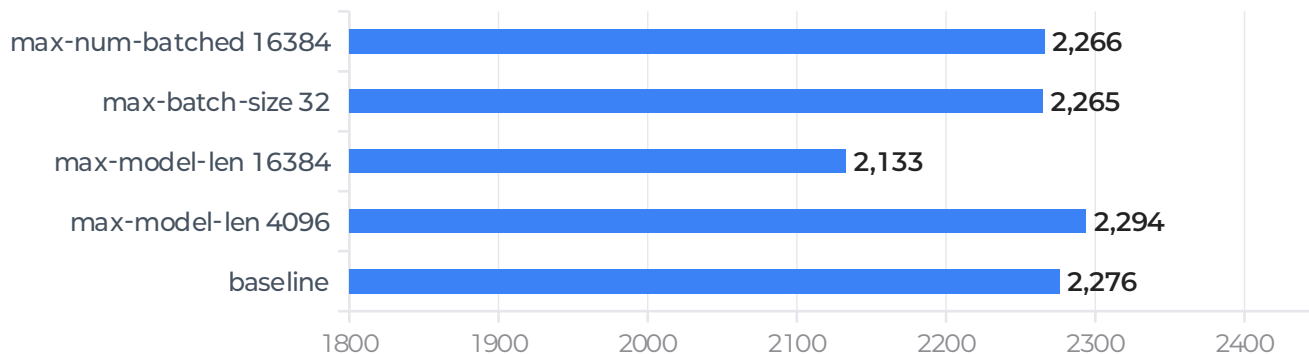
baseline + max-model-len·max-batch-size·max-num-batched-tokens 한 축씩 변경, 조합마다 서버 재기동.

출력·분석

조합별 합산 처리량 비교 → 어떤 옵션이 유의미한지 판단.

결과

serve 옵션 조합별 합산 처리량 (tok/s)



튜닝 효과 미미

Llama-3.1-8B 는 어떤 serve 옵션을 바꿔도 합산 처리량이 2,200 tok/s 대 (전체 범위 2,133~2,294, 편차 ~7%) 에서 거의 변하지 않는다 . KV cache·배치 한도 옵션을 손댈 실익이 없으며 기본값 그대로 서빙하면 된다 — 운영 시 튜닝 부담이 없다는 점은 장점 .

테스트 4 — SWE-bench ①: 정의 & 종류

GitHub 이슈를 패치로 해결하고 실제 repo 테스트 통과 여부로 채점하는 벤치마크

채점 방식

1 입력

GitHub 이슈 본문 + 해당 시점 repo 코드



2 모델 출력

이슈를 해결하는 unified diff (패치)



3 적용

Docker 컨테이너에서 repo 에 패치 적용



4 채점

테스트 실행 — FAIL→PASS 통과 시 resolved

데이터셋 종류

SWE-bench (full)

2,294 건 · 원본

SWE-bench Lite

300 건 · 저비용 평가용 subset

SWE-bench Verified

500 건 · 사람 검증, 신뢰도 최상

SWE-bench Multimodal

517 건 · 시각 요소 포함

본 측정

SWE-bench Lite · oracle · single-shot

Lite 300 건 중 12 개 repo(astropy·django·matplotlib·sympy 등) 에 고르게 50 건 추출 .
context 는 oracle(정답 파일 제공), 호출은 single-shot(1 회 생성) — 에이전트 반복 없이 순수 코드 편집 능력을 본다 .

테스트 4 — SWE-bench ②: 환경 구축

Docker 기반 채점 — 인스턴스별 격리 컨테이너에서 repo 테스트 실행

① 설치

```
$ git clone https://github.com/SWE-bench/SWE-bench.git
$ cd SWE-bench && pip install -e .
$ docker --version # Docker 동작 확인
```

② 채점 실행

```
$ python -m swebench.harness.run_evaluation \
  --dataset_name princeton-nlp/SWE-bench_Lite \
  --predictions_path predictions.jsonl \
  --max_workers 8 \
  --namespace swebench \
  --run_id my_run
# --namespace swebench → prebuilt 이미지 pull
# 결과: <model>.<run_id>.json (resolved 카운트)
```

요구사항

- x86_64 Linux · Docker
- 디스크 수심 ~120GB (인스턴스 이미지)
- Python 3.10+ · swebench 패키지

예측 파일 (predictions.jsonl)

한 줄 = 한 인스턴스. 모델이 만든 패치를 model_patch에 담는다:

```
{"instance_id": "astropy____"
 "model_name_or_path": "8B",
 "model_patch": "diff --git ..."}

```

테스트 4 — SWE-bench ③: 측정 방법

oracle 프롬프트를 로컬 furiosa-llm 서버에 보내 diff 를 받고 harness 로 채점

입력

oracle 프롬프트

이슈 설명 + 수정 대상 파일 코드

```
<issue>
separability_matrix 가
nested CompoundModel 에서
분리성을 잘못 계산함
</issue>

[start of separable.py]
def _separable(transform):
    ...
```

출력

모델 생성 diff

Llama-3.1-8B single-shot 응답

```
--- a/.../separable.py
+++ b/.../separable.py
@@ -304,6 +304,10 @@
     elif isinstance(
         transform, Compound):
+   + if isinstance(
+   +     transform.left,...):
+   +     sepleft =
+   +     _separable(...)
```

채점

Docker harness

패치 적용 후 테스트 실행

resolved

패치 적용 + 테스트 통과

unresolved

적용됐으나 테스트 미통과

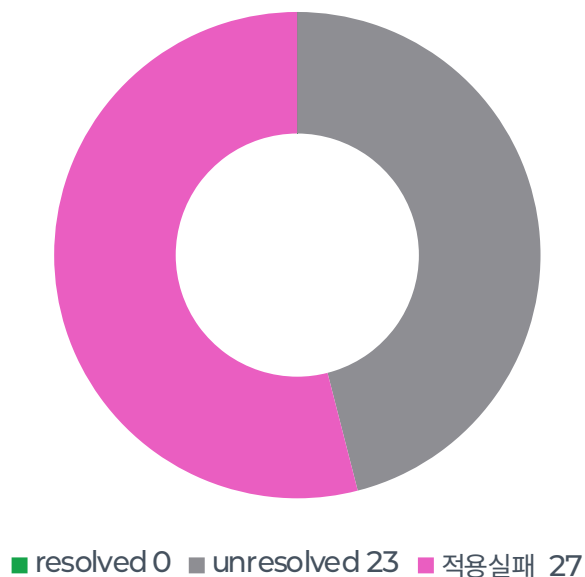
적용실패

malformed diff — 적용 불가

테스트 4 — SWE-bench ④: 결과

Llama-3.1-8B single-shot · oracle 컨텍스트 50 건 채점 결과

채점 결과 분포 (50 건)



repo 분포: astropy·django·matplotlib 각 5 · 기타 9 개 repo — Lite 12 개 repo stratified

resolved (테스트 통과)

0 / 50

8B 단발 생성으로는 SWE-bench 버그를 해결하지 못함 — 원인은 다음 슬라이드

27 건 — 적용실패

모델이 정확한 unified diff 를 만들지 못함 . diff 포맷이 무너진 비율 54%.

23 건 — 미해결

패치는 적용됐으나 테스트 미통과 . 단발 8B 코드 수정 역량의 한계 .

테스트 4 — SWE-bench ⑤: 결과 심층 분석

27 적용실패 · 23 미해결 — 모델 한계인가, NPU 한계인가?

결론

원인은 single-shot 8B 모델의 한계 — NPU·서빙 제약이 아니다

왜 NPU·서빙 한계가 아닌가

8B 아티팩트 = bf16

양자화 없음 → NPU 정밀도 손실 0

평균 출력 698 토큰

max_tokens 4096 한도 — 절단은 27 건 중 2 건뿐

context 128K

프롬프트 (5~18K 토큰) 충분히 수용 — 길이 제약 없음

두 실패 유형 — 모두 모델 역량 문제

27 적용실패

모델이 diff의 @@ hunk 헤더 줄 수를 잘못 계산 → patch 도구가 거부 (헤더는 +1 줄 표기, 실제 본문은 2 줄 추가 → 불일치).

23 미해결

문법이 맞는 패치가 적용됐으나 테스트 미통과 — 수정 위치·내용이 틀림. 단발 8B의 코드 추론 능력 한계.

시사점 정확도는 모델이 결정, 속도는 하드웨어가 결정 — 같은 모델이면 GPU에서도 SWE-bench 결과는 비슷할 것. NPU↔GPU 비교의 핵심은 속도·처리량.

테스트 5 — 임베딩 · 리랭커

검색 보조 모델 (Qwen3-Embedding-8B · Qwen3-Reranker-8B) 의 처리량 측정

무엇을

임베딩 / 리랭킹 처리량 — SWE-bench 검색 보조에 쓰이는 모델 .

입력

/v1/embeddings/v1/rerank 에 batch {1·4·16·64} 개 입력 .

출력 · 분석

초당 처리 건수 (inputs/s), 배치 크기별 효율 .

결과

Qwen3-Embedding-8B

1.17 inputs/s

batch 1·16·64 모두 동일 — 배치 이득 없음

Qwen3-Reranker-8B

1.17 pairs/s

쿼리당 100 문서 ≈ 85 초 — 항목당 약 0.85 초

처리량 이상 — 별도 점검 필요

두 모델 모두 항목당 약 0.85 초 — 8B 생성 모델의 단일 prefill(수십 ms) 대비 수십 배 느리다 . batch size 를 키워도 처리량이 1.17/s 로 고정돼 배치가 동작하지 않는다 .
아티팩트의 고정 버킷 패딩 또는 배칭 경로 미최적화 가능성 →
현재 설정으로는 대량 검색에 부적합하며 후속 점검이 필요하다 .

결론 — 2- 카드에서는 Llama-3.1-8B 단독

강력 후보 32B·70B 는 RNGD 4 카드 필요 → 정확도 우선이면 4 카드 환경 권장

동시 접속자별 권장 (Llama-3.1-8B · 1 카드)

동시 사용자	요청당 속도	TTFT p95	판단
1 - 8	48 - 54 tok/s	< 0.2 s	쾌적
16	42 tok/s	0.38 s	양호
32	35 tok/s	0.51 s	양호 · 1 카드 권장 상한
64	27 tok/s	0.96 s	처리량 우선
128	18 tok/s	1.85 s	집계 peak · 체감 느림

전 동시성 구간 요청 실패 0 건 · KV cache 27GB 확보로 KV 는 병목 아님

채택

Llama-3.1-8B / 1 카드

코드 자동완성 · 대화형 보조에 적합 . ~32 명까지 쾌적 , serve 옵션은 기본값 .

보강

32B·70B 는 4 카드에서

정확도 (SWE-bench) 는 8B 가 약함 . 강력 후보는 4 카드 환경에서 측정 · 운영 .

점검

임베딩 · 리랭커 처리량

1.17/s 는 비정상 — 대량 검색 전 배칭 / 버킷 설정 점검 필요 .